Uninformed & Informed Search Algorithms

Course Handout

AI 500: Foundations of AI

Contents

1 Introduction **4**
- 1.1 State Space Representation . . . 4
- 1.2 General Search Process . . . 4
- 1.3 Evaluation Properties of Search Algorithms . . . 5

2 Uninformed Search Algorithms **6**
- 2.1 Breadth-First Search (BFS) . . . 6
  - 2.1.1 Algorithm . . . 6
  - 2.1.2 Complexity Analysis . . . 7
  - 2.1.3 Advantages and Disadvantages . . . 7
  - 2.1.4 Exercises . . . 9
- 2.2 Depth-First Search (DFS) . . . 10
  - 2.2.1 Algorithm . . . 10
  - 2.2.2 Complexity Analysis . . . 10
  - 2.2.3 Advantages and Disadvantages . . . 11
  - 2.2.4 Exercises . . . 12
- 2.3 Iterative Deepening Depth-First Search (IDDFS) . . . 13
  - 2.3.1 Algorithm . . . 13
  - 2.3.2 Complexity Analysis . . . 13
  - 2.3.3 Advantages and Disadvantages . . . 13
  - 2.3.4 Exercises . . . 14

2.4 Bidirectional Search . . . 15
2.4.1 Properties . . . 15
2.4.2 Advantages and Disadvantages . . . 15
2.4.3 Exercises . . . 16

3 Informed (Heuristic) Search Algorithms 16
3.1 Heuristic Functions . . . 16
3.1.1 Properties of Heuristic Functions . . . 16
3.1.2 Exercises . . . 17
3.2 Best-First Search . . . 17
3.2.1 Algorithm . . . 17
3.2.2 Exercises . . . 17
3.3 Greedy Best-First Search . . . 17
3.3.1 Properties . . . 17
3.3.2 Advantages and Disadvantages . . . 18
3.3.3 Exercises . . . 19
3.4 A* Algorithm . . . 22
3.4.1 Evaluation Function . . . 22
3.4.2 Algorithm . . . 23
3.4.3 Admissibility Properties . . . 23
3.4.4 Properties of A* . . . 23
3.4.5 Advantages and Disadvantages . . . 24
3.4.6 Exercises . . . 25

4 Problem Reduction Search 26
4.1 AND-OR Graphs . . . 26
4.2 AO* Algorithm . . . 27
4.2.1 Algorithm . . . 27
4.2.2 Cost Revision Process . . . 27
4.2.3 Advantages . . . 28
4.2.4 Exercises . . . 29

5 Memory-Bounded Heuristic Search **30**
5.1 Iterative Deepening A* (IDA*) . . . 30
5.1.1 Algorithm . . . 31
5.1.2 Properties . . . 31
5.1.3 Exercises . . . 31
5.2 Recursive Best-First Search (RBFS) . . . 31
5.2.1 Properties . . . 31
5.2.2 Advantages and Disadvantages . . . 32
5.2.3 Exercises . . . 32

6 Comparison of Search Algorithms **32**

7 Exercises **34**
7.1 Part 1 . . . 34
7.2 Part 2 . . . 36

8 Solutions **37**
8.1 Part 1 . . . 37
8.2 Part 2 . . . 38

9 Study Resources **41**

1 Introduction



Fig 1: A sequence of action in a search space from initial to goal state.

In Artificial Intelligence, many problems can be formulated as search problems. Before an AI problem can be solved, it must be represented as a state space. The process of searching means finding a sequence of actions that take you from an initial state to a goal state.

Search algorithms are one of the most important areas of Artificial Intelligence. This unit explains all about the search algorithms in AI which explore the search space to find a solution. In Artificial Intelligence, search techniques are universal problem-solving methods used by rational agents or problem-solving agents to solve specific problems and provide the best results.

1.1 State Space Representation

A state space is a graph (V, E) where V is a set of nodes and E is a set of arcs, where each arc is directed from one node to another node.

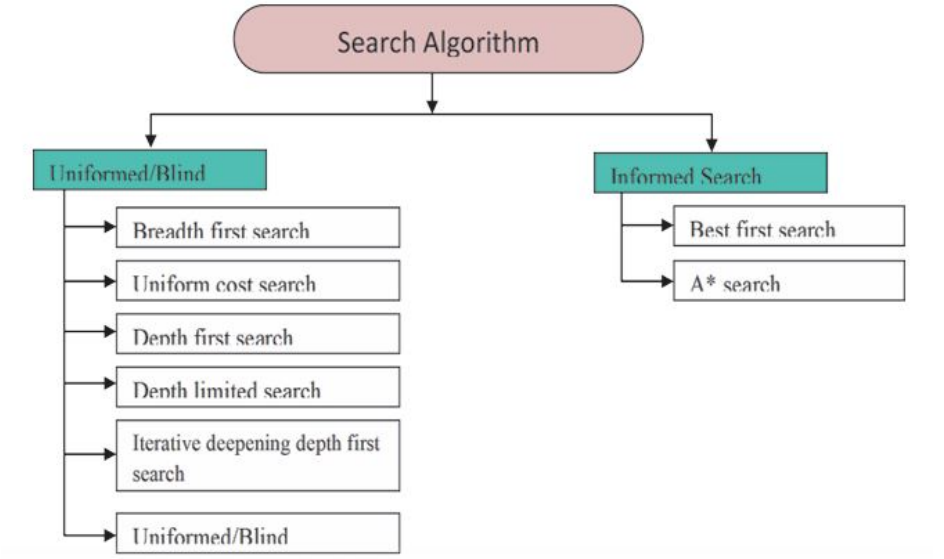
- **V**: A node is a data structure that contains state description, plus optionally other information related to the parent of the node, operation to generate the node from that parent, and other bookkeeping data.
- **E**: Each arc corresponds to an applicable action/operation. The source and destination nodes are called parent (immediate predecessor) and child (immediate successor) nodes with respect to each other.

Each node has a set of successor nodes corresponding to all operators that can apply at the source node's state. Expanding a node means generating successor nodes and adding them to the state-space graph.

1.2 General Search Process

The searching process starts from the initial state (root node) and proceeds by performing the following steps:

1. Check whether the current state is the goal state or not
2. Expand the current state to generate new sets of states
3. Choose one of the new states generated for search depending upon search strategy
4. Repeat steps 1 to 3 until the goal state is reached or there are no more states to be expanded



1.3 Evaluation Properties of Search Algorithms

Any search algorithm is commonly evaluated according to the following four criteria:

- **Completeness**: A search algorithm is complete if it guarantees to return a solution if one exists.
- **Optimality/Admissibility**: If a solution found by an algorithm is guaranteed to be the best solution (lowest path cost) among all other solutions, then such a solution is said to be optimal.
- **Time Complexity**: Time complexity is a measure of time for an algorithm to complete its task, usually measured in terms of the number of nodes expanded during the search.
- **Space Complexity**: It is the maximum storage space required at any point during the search, usually measured in terms of the maximum number of nodes in memory at a time.

Time and space complexity are measured in terms of:

- b - maximum branching factor of the search tree

- d - depth of the least-cost solution
- m - maximum depth of the search tree (may be infinity)

Uninformed Search	Informed Search
No information about the path, cost, from the current state to the goal state. It doesn't use domain specific knowledge for searching process.	The path cost from current state to goal state is calculated, to select the minimum cost path as the next state. It uses domain specific knowledge for the searching process.
It finds solution slow as compared to informed	It finds solution more quickly.
Less efficient	More efficient
Cost is high.	Cost is low
No suggestion is given regarding the solution init. Problem to be solved with the given information only.	It provides the direction regarding the solution. Additional information can be added as assumption to solve the problem.

2 Uninformed Search Algorithms

Uninformed search is also called brute force search or blind search. It is called blind search because of the way in which the search tree is searched without using any information about the search space. It assumes no additional knowledge other than how to traverse the search tree and how to identify leaf nodes and goal nodes.

2.1 Breadth-First Search (BFS)

Breadth-First Search is the simplest form of blind search. In this technique, the root node is expanded first, then all its successors are expanded, and then their successors, and so on. All nodes are expanded at a given depth in the search tree before any nodes at the next level are expanded.

2.1.1 Algorithm

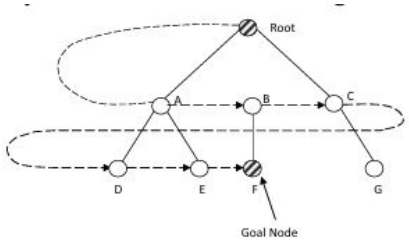


Fig 2 Search tree for BFS

Algorithm 1 Breadth-First Search

1: Initialize: Set OPEN = {s}, where s is the start state
2: Fail: If OPEN = {}, terminate with failure
3: Select: Remove leftmost state from OPEN
4: Terminate: If selected state is goal, terminate with success
5: Expand: Generate successors, discard if already in OPEN, insert remaining successors at right end of OPEN
6: LOOP: Go to Step 2

2.1.2 Complexity Analysis

- **Time Complexity**: $O(b^d)$
- **Space Complexity**: $O(b^d)$
- **Complete**: Yes (if b is finite)
- **Optimal**: Yes (if cost = 1 per step)

2.1.3 Advantages and Disadvantages

Advantages:

- BFS will never get trapped exploring a blind alley
- It is guaranteed to find a solution if one exists

Disadvantages:

- Time and space complexity are both exponential
- All nodes must be generated, including unwanted nodes

2.1.4 Exercises

Example1: Consider the following graph in fig-1 and its corresponding state space tree representation in fig-2. Note that A is a start state and G is a Goal state.

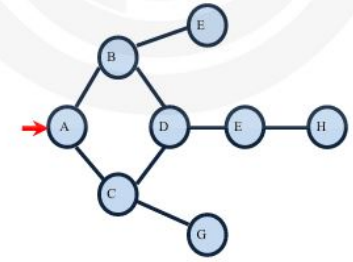


Fig-1 State space graph

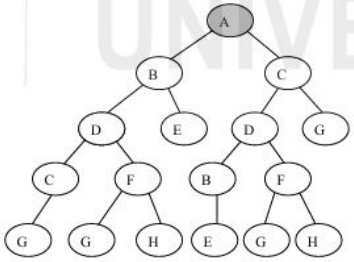
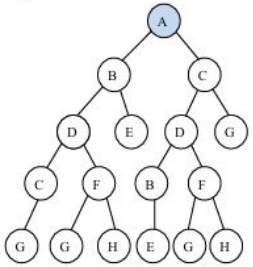


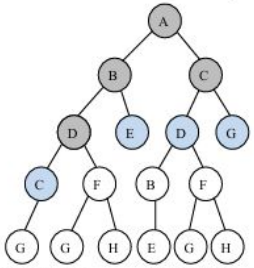
Fig-2 State space tree

Step1: Initially open contains only one node corresponding to the source state A.



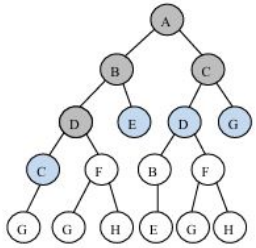
Open= A CLOSED={ }

Step5: Node D is removed from open. Its children C and F are generated and added to the back of open.



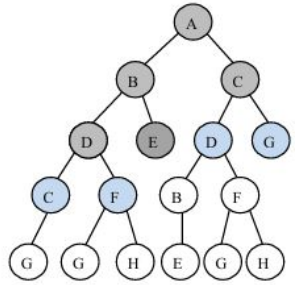
Open= E,D,G,C,F CLOSED=A,B,C,D

Step2: A is removed from open. The node is expended, and its children B and C are generated. They are placed at the back of open.



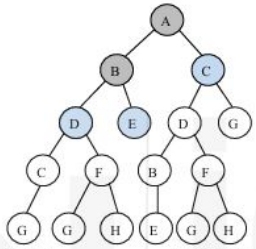
Open= B,C CLOSED={A}

Step 6: Node E is removed from open. It has no children



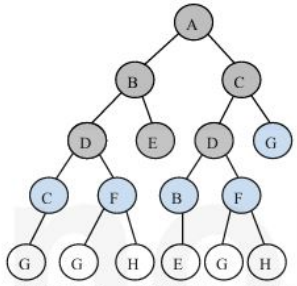
Open=D,G,C,F CLOSED=A,B,C,D,E

Step: 3: Node B is removed from open and is expended. Its children D, E are generated and put at the back of open.



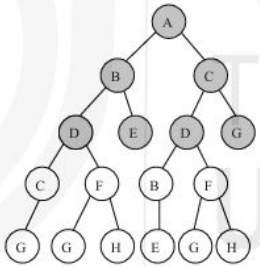
Open= C,D,E CLOSED={A,B}

Step 7: D is expanded, B and F are put in OPEN.



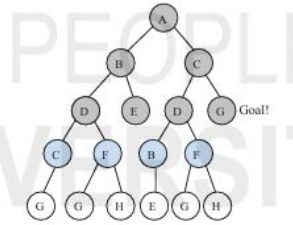
Open=G,C,F,B,F CLOSED=A,B,C,D,E,D

Step 4: Node C is removed from open and is expanded its children D and G are added to the back of open.



Open= D,E,D,G CLOSED={A,B,C}

Step 8: G is selected for expansion. It is found to be a goal node. So, the algorithm returns the path ACG by following the parent pointers of the node corresponding to G. The algorithm terminates.



Open=C,F,B,F CLOSED={A,B,C,D,E.G}

2.2 Depth-First Search (DFS)

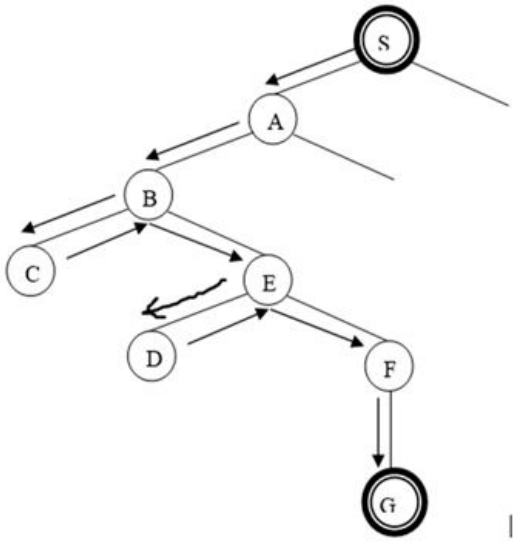


Fig 4 Depth first search (DFS) tree

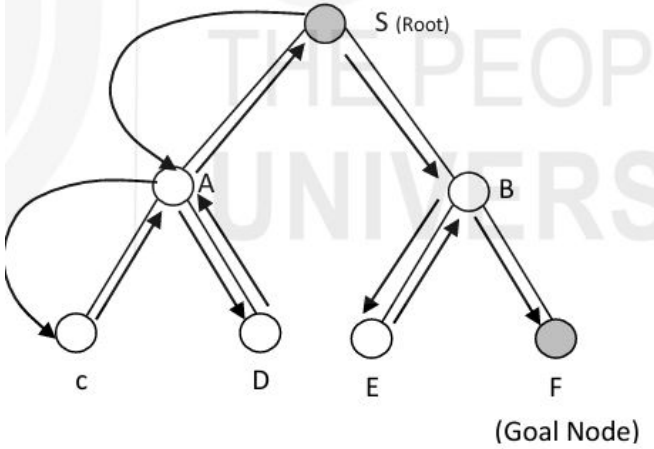


Fig. 5 Search tree for DFS

Depth-First Search explores a path all the way to a leaf before backtracking and exploring another path. It expands the deepest unexpanded node first.

2.2.1 Algorithm

2.2.2 Complexity Analysis

- **Time Complexity**: $O(b^m)$
- **Space Complexity**: $O(bm)$
- **Complete**: No (can get stuck in infinite loops)

Algorithm 2 Depth-First Search

1: Initialize: Set OPEN = {s}, where s is the start state
2: Fail: If OPEN = {}, terminate with failure
3: Select: Remove leftmost state from OPEN
4: Terminate: If selected state is goal, terminate with success
5: Expand: Generate successors, discard if already in OPEN, insert remaining successors at left end of OPEN (STACK)
6: LOOP: Go to Step 2

- **Optimal**: No

2.2.3 Advantages and Disadvantages

Advantages:

- Memory requirement is linear with respect to search depth
- If solution is found without exploring much, time and space requirements are low

DFS: The problem with this appraoch is, if there is a node close to root, but not in first few subtrees explored bt DFS, then DFS reaches that node very late. Also, DFS may not find shortest path to a node (in terms of number of edges.)

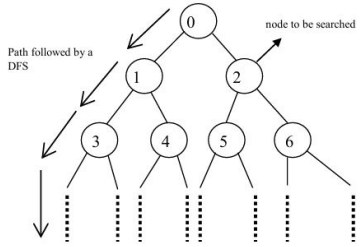


Fig 6: Path sequence in DFS

Disadvantages:

- Can get trapped in infinite loops
- May not find optimal solution
- Can take very long if solution is at shallow depth but not in early subtrees

2.2.4 Exercises

Example1: Consider the following graph in fig-1 and its corresponding state space tree representation in fig-2. Note that A is a start state and G is a Goal state.

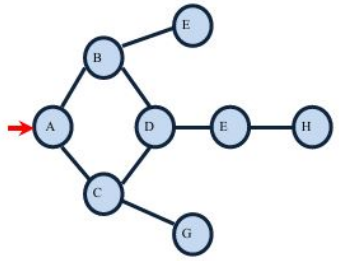


Fig-1 State space graph

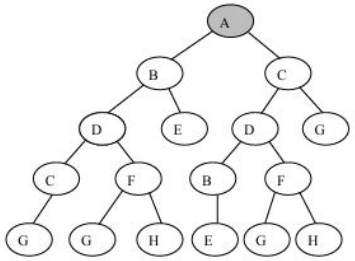
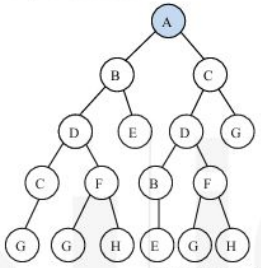


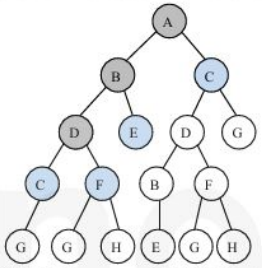
Fig-2 State space tree

Step 1: Initially open contains only one node corresponding to the source state A.



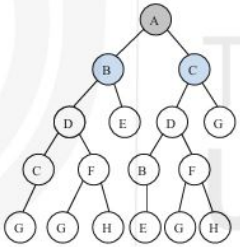
Open= A CLOSED={ }

Step 4: Node D is removed from open and is expanded. Its children C and F are added to the front of open.



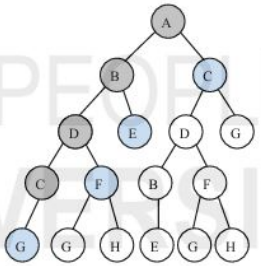
Open= C,F,E,C CLOSED=A,B,D

Step2: A is removed from open. The node A is expanded, and its children B and C are generated. They are placed at the from of open.



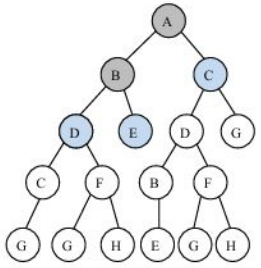
Open= B,C CLOSED={A}

Step 5: Node C is removed from open. Its children G and F is added to the front of open.



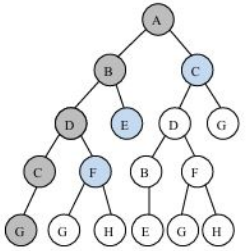
Open= G,F,E,C CLOSED=A,B,D,C

Step 3: Node B is removed from open and is expended. Its children D, E are generated and put at the front open.



Open= D,E,C CLOSED={A,B}

Step 6: Node G is expanded and found to be a goal node. The solution path A-B-D-C-G is returned and the algorithm terminates.



Open= F,E,C CLOSED=A,B,D,C,G

2.3 Iterative Deepening Depth-First Search (IDDFS)

IDDFS combines the benefits of both BFS and DFS. It begins by performing DFS to depth zero, then depth one, then depth two, and so on until a solution is found or some maximum depth is reached.

2.3.1 Algorithm

Algorithm 3 Iterative Deepening DFS

```
Initialize d = 1, found = false
while found = false do
    Perform depth-first search from start to depth d
    if goal state is obtained then
        found = true
    else
        Discard nodes generated beyond depth d
        d = d + 1
    end if
end while
Report solution if obtained
```

2.3.2 Complexity Analysis

- **Time Complexity**: $O(b^d)$
- **Space Complexity**: $O(bd)$
- **Complete**: Yes
- **Optimal**: Yes (if path cost is non-decreasing function of depth)

2.3.3 Advantages and Disadvantages

Advantages:

- Combines benefits of BFS and DFS
- Guaranteed to find shortest path solution
- Preferred when search space is large and depth of solution is unknown

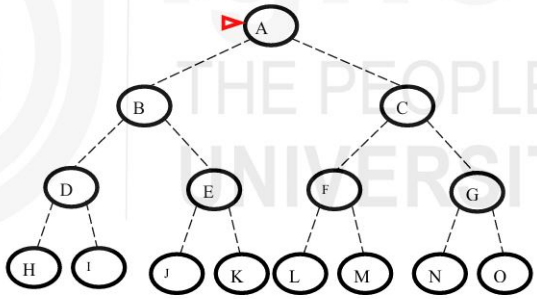
Disadvantages:

- Performs wasted computation before reaching goal depth
- Time complexity remains exponential

2.3.4 Exercises

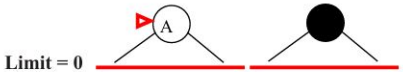
Let us consider the following example to understand the IDDFS:

- Here initial state is A and goal state is M :

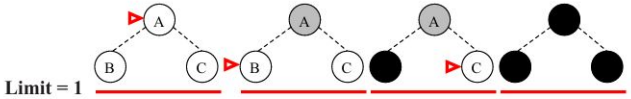


The Iterative deepening search proceeds as follows:

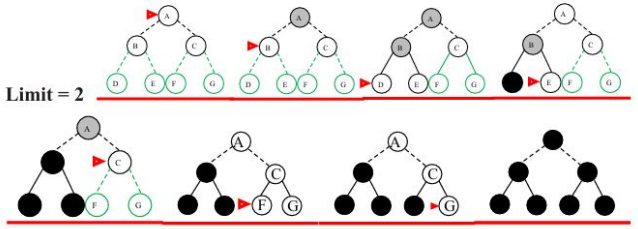
Iterative Deepening search L=0



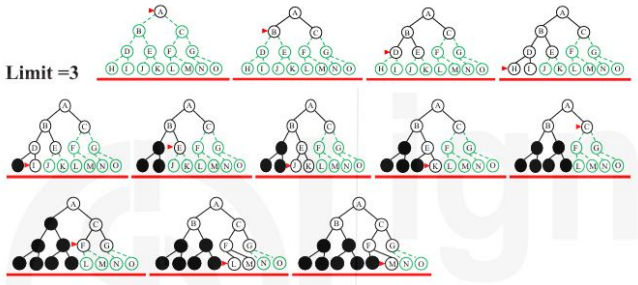
Iterative Deepening search L=1



Iterative Deepening search L=2



Iterative Deepening search L=3



2.4 Bidirectional Search

Bidirectional search expands nodes from both the start and goal states simultaneously, checking at each stage if nodes from one direction have been generated by the other direction.

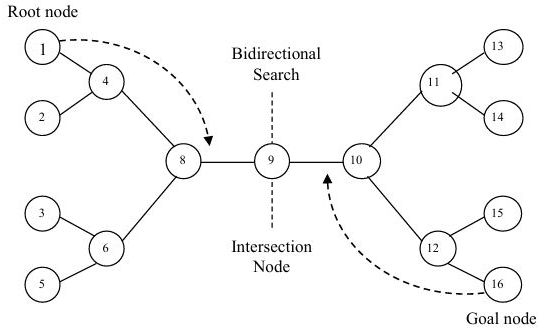


Fig 7 Bidirectional search

2.4.1 Properties

- **Time Complexity**: $O(b^{d/2})$
- **Space Complexity**: $O(b^{d/2})$
- **Complete**: Yes
- **Optimal**: Yes (if step costs are uniform)

2.4.2 Advantages and Disadvantages

Advantages:

- More efficient than other uninformed search methods
- Requires less memory

Disadvantages:

- Implementation is more complex
- Goal state must be known in advance
- Additional overhead for checking intersection points

2.4.3 Exercises

3 Informed (Heuristic) Search Algorithms

Informed search algorithms use domain-dependent heuristic information beyond the definition of the problem itself to search the space more efficiently. These algorithms have additional information that can guide the search.

The following 2 list (**OPEN** and **CLOSED**) are maintained to implement these two algorithms.

1. **OPEN** – all those nodes that have been generated & have has heuristic function applied to them but have not yet been examined.
2. **CLOSED** – contains all nodes that have already been examined.

3.1 Heuristic Functions

A heuristic function $h(n)$ estimates the cost of the optimal path from state n to a goal state. The heuristic function is an estimate based on domain-specific information that is computable from the current state description.

3.1.1 Properties of Heuristic Functions

- $h(n) = 0$ if n is a goal node
- $h(n) \geq 0$ for all nodes n
- Heuristic should be easy to compute
- Should provide good guidance toward goal

3.1.2 Exercises

3.2 Best-First Search

Best-first search uses an evaluation function $f(n)$ for each node that gives an indication of which node to expand next. The decision of which node to expand depends on the value of the evaluation function.

3.2.1 Algorithm

Algorithm 4 Best-First Search

1: Initialize: Set OPEN = {s}, CLOSED = {}, $f(s) = h(s)$
2: Fail: If OPEN = {}, terminate with failure
3: Select: Select minimum cost state from OPEN, move to CLOSED
4: Terminate: If selected state is goal, terminate with success
5: Expand: For each successor m of selected node, if $m \notin$ [OPEN $\cup$ CLOSED], set $f(m) = h(m)$ and insert m in OPEN
6: LOOP: Go to Step 2

3.2.2 Exercises

3.3 Greedy Best-First Search

Greedy best-first search always selects the path that appears best at the moment. It uses the heuristic function directly as the evaluation function: $f(n) = h(n)$.

3.3.1 Properties

- **Time Complexity**: $O(b^m)$ (worst case)
- **Space Complexity**: $O(b^m)$ (worst case)
- **Complete**: No (can get stuck in loops)
- **Optimal**: No

3.3.2 Advantages and Disadvantages

Advantages:

- Can switch between BFS and DFS behavior
- More efficient than uninformed search methods

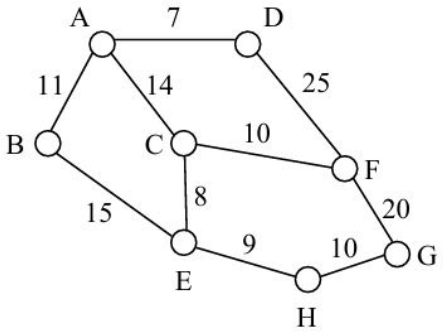
Disadvantages:

- Can get stuck in loops
- Can behave like unguided depth-first search in worst case

3.3.3 Exercises

Example1: Consider the following **example** (graph) with heuristic function value h(n)[Fig 2] which illustrate the greedy Best-first search. Note that in the following example, heuristic function is defined as

$\mathbf{h_{SLD}}$ = straight line distance from n to goal



Let heuristic function value h(n) for each node n to goal node G is defined as

Straight line distance

A→G=h(A)=40

B→G=h(B)=32

C→G=h(C)=25

D→G=h(D)=35

E→G=h(E)=19

F→G=h(F)=17

H→G=h(H)=10

G→G=h(G)=0

h(n)= straight line distance from node n to G

Note that h(G)=0

The nodes added/deleted from OPEN and CLOSED list using Best-First Search algorithm are shown below.

OPEN	CLOSED
[A]	[ ]
[C,B,D]	[A]
B,D	A,C
F,E,B,D	A,C
G.E.B.D	A,C,F
E,B,D	A,C,F,G

Explanation are as follows:

Step1: initially OPEN list start with start state 'A' and CLOSED list with empty.

Step2: Children of A={C[25], B[32] D[35]}, so

OPEN={C[25], B[32], D[35]} therefore Best=C, so expend C node next.

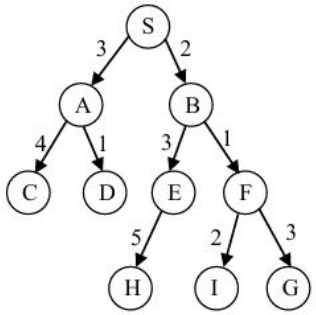
Step3: Children of C={E[19],F[17]}, so

OPEN={F[17],E[19], B[32], D[35]} therefore Best=F, so expend node F next.

Step4: Children of F={G[0]}, therefore OPEN= {G[0], E[19], B[32], D[35]} Best=G, this is a goal node so Stop.

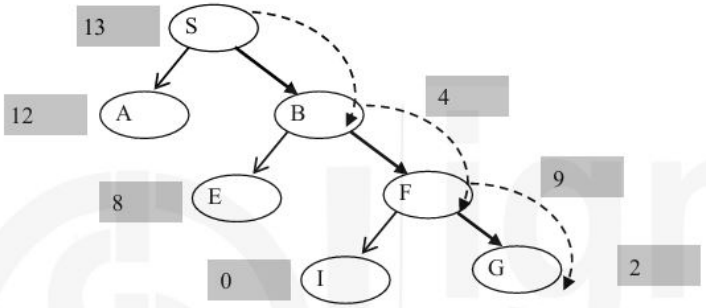
Finally, we got the shortest path: A→C→F→G and cost is 44.

Example2: Consider the below search problem, and we will traverse it using greedy best-first search. At each iteration, each node is expanded using evaluation function f(n)=h(n) , which is given in the below table.



node	h(n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13

Here, we are using two lists which are **OPEN** and **CLOSED** Lists. Following are the iteration for traversing the above example.



Initialization: Open [A, B], Closed [S]

Iteration 1 : Open [A], Closed [S, B]

Iteration 2 : Open [E, F, A], Closed [S, B]
: Open [E, A], Closed [S, B, F]

Iteration 3 : Open [I, G, E, A], Closed [S, B, F]
: Open [I, E, A], Closed [S, B, F, G]

Hence the final solution path will be **S----> B-----> F----> G**

Example3: Apply Greedy Best-First Search algorithm on the following graph (L is a goal node).

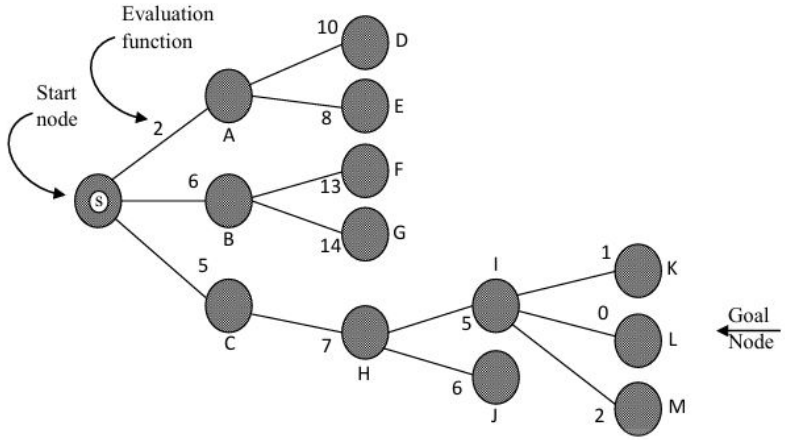


Fig. 2.17: Greedy best first search graph.

Working: We start with a start-nodes, S. Now, S has three children i.e., A, B and C with their Heuristic function values 2, 6 and 5 respectively. These weights show approximately, how far theyare from goal node. So, we write, children of S are -(A:2), (B: 6), (C:5)

Out of these, the node with minimum value is (A : 2). So, we select A and its children are explored (or generated).

Its children are(D: 10) and (E:8)

The search process now has four nodes to search for, namely-

(B : 6), (C: 5), (8.10) and (E: 8)

Out of these, node-C has the minimal value of 5. So, we select it and expand. So, we get(H: 7) as its child. Now, the nodes to search are as follows-

(B:6), (D: 10,(E: 8) and (H : 7) and so on.

Working of the algorithm can be represented in tabular firm as follows

Step	Node being expanded	Children (on expansion)	Available nodes (to search)	Node Chosen
1.	S	(A:2), (B:6), (C:5)	(A:2), (B:6), (C:5)	(A:2)
2.	A	(D:10), (E:8)	(B:6), (C:5), (D:10), (E:8)	(C:5)
3.	C	(H:7)	(B:6), (D:10), (E:8), (H:7)	(B:6)
4.	B	(F:13), (G:14)	(D:10), (E:8), (H:7), (F:13), (G:14)	(H:7)
5.	H	(I:5), (J:6)	(D:10), (E:8), (F:13), (G:14), (I:5), (J:6)	(I:5)
6.	I	(K:1), (L:0), (M:2)	(D:10), (E:8), (H:7), (F:13), (G:14), (J:6), (K:1), (L:0), (M:2)	Goal node is found. So, search stops now

3.4 A* Algorithm

A* search is the most commonly known form of best-first search. It uses both the heuristic function $h(n)$ and the cost to reach the node $g(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

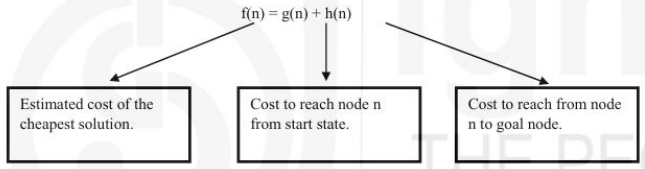


Fig 9 Evaluation function f(n) in A* Algorithm

3.4.1 Evaluation Function

- $g(n)$ = actual cost from start state to node n
- $h(n)$ = estimated cost from node n to goal
- $f(n) = g(n) + h(n)$ = estimated cost of cheapest solution through n

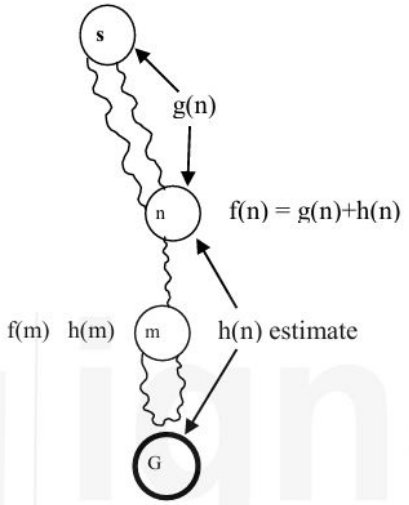


Fig 10 Calculation of evaluation function f(n).

3.4.2 Algorithm

3.4.3 Admissibility Properties

A heuristic is called admissible if it never overestimates the actual cost, i.e., $h(n) \leq h^*(n)$ where $h^*(n)$ is the true cost from n to the goal.

Admissibility Conditions:

1. Each node has finite number of successors
2. All arc costs are greater than some positive amount
3. For each node n, $h(n) \leq h^*(n)$

3.4.4 Properties of A*

- **Complete**: Yes (if admissible heuristic)
- **Optimal**: Yes (if admissible heuristic)

Algorithm 5 A* Search

1: Initialize: Set OPEN = {s}, CLOSED = {}, $g(s) = 0$, $f(s) = h(s)$
2: Fail: If OPEN = {}, terminate with failure
3: Select: Select minimum cost state n from OPEN, move n to CLOSED
4: Terminate: If n is goal, terminate with success
5: Expand: For each successor m of n:
6: **if** $m \notin$ [OPEN $\cup$ CLOSED] **then**
7: Set $g(m) = g(n) + c(n, m)$, $f(m) = g(m) + h(m)$, insert m in OPEN
8: **end if**
9: **if** $m \in$ [OPEN $\cup$ CLOSED] **then**
10: Set $g(m) = \min\{g(m), g(n) + c(n, m)\}$, $f(m) = g(m) + h(m)$
11: **if** $f(m)$ decreased and $m \in$ CLOSED **then**
12: Move m to OPEN
13: **end if**
14: **end if**
15: LOOP: Go to Step 2

- **Optimally Efficient**: Yes (expands fewest nodes)

3.4.5 Advantages and Disadvantages

Advantages:

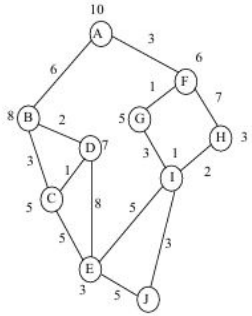
- Optimal and complete with admissible heuristic
- Can solve complex problems efficiently
- Best algorithm among search algorithms

Disadvantages:

- Keeps all generated nodes in memory
- Not practical for large-scale problems due to memory requirements
- Performance depends on quality of heuristic

3.4.6 Exercises

Example1: Let's us consider the following graph to understand the working of A* algorithm. The numbers written on edges represent the distance between the nodes. The numbers written on nodes represent the heuristic value. Find the most cost-effective path to reach from start state **A** to **final** state **J** using A* Algorithm.



Step-1:

We start with node A. Node B and Node F can be reached from node A. A* Algorithm calculates f(B) and f(F). Estimated Cost f(n) = g(n) +h(n) for Node B and Node F is:

f(B) = 6+8=14

f(F) = 3+6=9

Since f(F) < f(F), so it decides to go to node F.

→ Closed list (F)

Path- A →F

Step-2: Node G and Node H can be reached from node F.

A* Algorithm calculates f(G) and f(H).

f(G) = (3+1)=5=9

f(H) = (3+7) +5=13

Since f(G) < f(H), so it decides to go to node G.

→ Closed list (G)

Path- A→F→G

Step-3: Node I can be reached from node G.

A* Algorithm calculates f(I).

f(I)=(3+1+3)+1=8; It decides to go to node I.

→ Closed list (I).

Path- A→F→G→I

Step-4: Node E, Node H and Node J can be reached from node I.

A* Algorithm calculates f(E), f(H), f(J).

f(E) = (3+1+3+5) + 3 =15

f(H) = (3+1+3+2) + 3 = 12

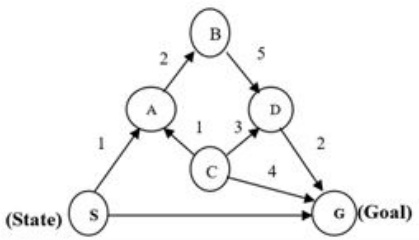
f(J) = (3+1+3+3) +0 = 10

Since f(J) is least, so it decides to go to node J.

→ Closed list (J)

Shortest Path - A→F→G→I→J

Path Cost is 3+1+3+3=10



State	h(n)
S	5
A	3
B	4
C	2
D	6
G	0

Example 2: Consider the following graph and apply A* algorithm and find the most cost-effective path to reach from start state S to final stateG. The heuristic function value of each node n is defined in the table given.

Solution:

S→A=1+3=4

S→G=10+0=10

S→A→B=1+2+4=7

S→A→C=1+1+2=4

S→A→C→D=1+1+3+6=11

S→A→C→G=1+1+4=6

S→A→B→D=1+2+5+6=14

S→A→C→D→G=1+1+3+2=7

S→A→B→D→G=1+2+5+2=10

4 Problem Reduction Search

Problem reduction search involves decomposing a problem into subproblems that can be solved independently. This approach uses AND-OR graphs where OR nodes represent choices between decompositions and AND nodes represent required decompositions.

4.1 AND-OR Graphs

In AND-OR graphs:

- **OR nodes**: Represent alternative ways to solve a problem
- **AND nodes**: Represent decomposition where all subproblems must be solved

- **AND arcs**: Connect a node to all nodes that must be solved together

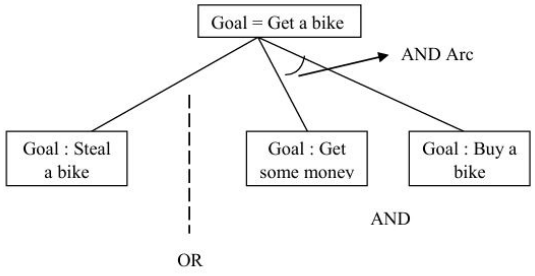


Fig 11AND-OR graph

4.2 AO* Algorithm

AO* algorithm is used for problem reduction search in AND-OR graphs. It finds the optimal solution by exploring the most promising paths first.

4.2.1 Algorithm

Algorithm 6 AO* Algorithm

1: Initialize: Set G = {s}, $f(s) = h(s)$. If s is terminal, label s as SOLVED
2: Terminate: If s is SOLVED, terminate
3: Select: Select non-terminal leaf node from marked subtree
4: Expand: Generate successors, set $f(m) = h(m)$ for each successor m
5: Cost Revision: Call Cost-Revise procedure
6: LOOP: Go to Step 2

4.2.2 Cost Revision Process

For AND nodes: $f(n) = \sum_i [f(r_i) + c(n, r_i)]$ where r_i are successors For OR nodes: $f(n) = \min_i \{f(r_i) + c(n, r_i)\}$ where r_i are successors

4.2.3 Advantages

- Finds minimum cost solution if admissible heuristic is used
- Guaranteed to terminate even with cycles
- Efficient for problems that can be decomposed

4.2.4 Exercises

Example1:

Let us see one example with the presence of heuristic value at every node (see fig 2) . The estimated heuristic value is given at each node. The heuristic value h(n) at any node indicates "from this node at least h(n) value (or cost) is required to find solution". Here we assume the edge cost value (i.e., g(n) value) for each edge is 1. Remember in OR node we always mark that successor node which indicates best path for solution.

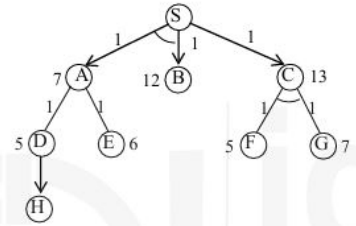


Fig 12: AND-OR graph with heuristic value at each node

Note that, the graph given in fig 2, there are two paths for solution from start state S: either S-A-B or S-C. To calculate the cost of the path we use the formula f(n)=g(n)+h(n) [note that here g(n) value is 1 for every edge].

Path1: f(S-A-B)=1+1+7+12=21

Path2: f(S-C)=1+13=14

Since min(21,14)=14; so, we select successor node C, as its cost is minimum, so it indicates best path for solution.

Note that C is a AND node; so, we consider both the successor node of C. The cost of node C is f(C-F-G)=1+1+5+7=14; so, the revised cost of node C is 14 and now the revised cost of node S is f(S-C)=1+14=15 (revised).

Note that once the cost (that is f value) of any node is revised, we propagate this change backward through the graph to decide the current best path.

Now let us explore another path and check whether we are getting lessor cost as compared to this cost or not.

F(A-D)=1+5=6 and f(A-E)=1+6=7; since A is an OR node so best successor node is D since min(6,7)=6. So revised cost of Node A will be 6 instead of 7, that is f(A)=6 (revised). Now next selected node is D and D is having only one node H so f(D-H)=1+2=3, so the revised cost of node D is 3, so now the revised cost of node A, that is f(A-D-H)=4. This path is better than f(A-E)=7. So, the final revised cost of node A is 4. Now the final revised cost of f(S-A-B)=1+1+4+12=18 (revised).

Thus, the final revised cost for

Path1: f(S-A-B)=18 and

Path2: f(S-C)=15

So optimal cost is 15.

5 Memory-Bounded Heuristic Search

Memory-bounded search algorithms address the space complexity issues of A* while maintaining optimality and completeness.

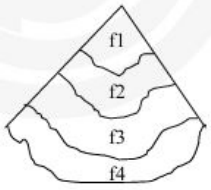
5.1 Iterative Deepening A* (IDA*)

- Perform depth-first search LIMITED to some f-bound.
- If goal found then ok.
- Else: increase the f-bound and restart.

How to establish the f-bound?

Initially: f(S)

- Generate all successors
- Record the minimal f(succ) > f(S)
- Continue with minimal f(succ) instead of f(S)



IDA* is similar to iterative deepening depth-first search but uses f-cost limits instead of depth limits.

5.1.1 Algorithm

Algorithm 7 IDA* Algorithm

1: Set C = $f(s)$
2: Perform depth-first branch-and-bound with cutoff C
3: Expand state n only if $f(n) \leq C$
4: **if** goal is selected for expansion **then**
5: Return C and terminate
6: **else**
7: Update C to minimum f-value that exceeded C
8: Go to Step 2
9: **end if**

5.1.2 Properties

- **Complete**: Yes
- **Optimal**: Yes
- **Space Complexity**: $O(bd)$
- **Time Complexity**: $O(b^d)$

5.1.3 Exercises

5.2 Recursive Best-First Search (RBFS)

RBFS simulates A* search with linear space complexity by using recursive calls and keeping track of alternative paths.

5.2.1 Properties

- **Space Complexity**: $O(bd)$
- **Complete**: Yes (with admissible heuristic)

- **Optimal**: Yes (with admissible heuristic)
- More efficient than IDA* but still suffers from node regeneration

5.2.2 Advantages and Disadvantages

Advantages:

- Linear space complexity
- More efficient than IDA*
- Optimal with admissible heuristic

Disadvantages:

- Still suffers from excessive node regeneration
- Difficult to analyze time complexity
- Not suitable for graphs with repeated states

5.2.3 Exercises

6 Comparison of Search Algorithms

Algorithm	Time Complexity	Space Complexity	Complete	Optimal	Notes
BFS	$O(b^d)$	$O(b^d)$	Yes	Yes*	*Optimal if step costs are uniform [**?**]
DFS	$O(b^m)$	$O(bm)$	No	No	May get trapped in infinite paths; space-efficient [**?**]

Algorithm	Time Complexity	Space Complexity	Complete	Optimal	Notes
UCS/Dijkstra	$O(b^{\lvert C^*/\epsilon \rvert})$ where C^* is optimal cost, ϵ is min edge cost	$O(b^{\lvert C^*/\epsilon \rvert})$	Yes	Yes	Optimal for weighted graphs; explores by path cost [**?**]
IDDFS	$O(b^d)$	$O(bd)$	Yes	Yes*	*Optimal if step costs are uniform; combines DFS space efficiency with BFS optimality
Bidirectional	$O(b^{d/2})$	$O(b^{d/2})$	Yes	Yes*	*If both searches use BFS; significant improvement over unidirectional search
Greedy Best-First	$O(b^m)$	$O(b^m)$	No	No	Uses heuristic $h(n)$; can be very efficient but not guaranteed [**?**]
A*	$O(b^d)$	$O(b^d)$	Yes	Yes	With admissible heuristic; exponential space is main limitation [**?**]
IDA*	$O(b^d)$	$O(bd)$	Yes	Yes	Iterative deepening version of A*; linear space complexity
RBFS	$O(b^d)$	$O(bd)$	Yes	Yes	Recursive best-first search; linear space, may re-expand nodes [**?**]

Legend

- b = branching factor (average number of successors per node)

- d = depth of optimal solution
- m = maximum depth of search space
- C^* = cost of optimal solution
- ϵ = minimum step cost

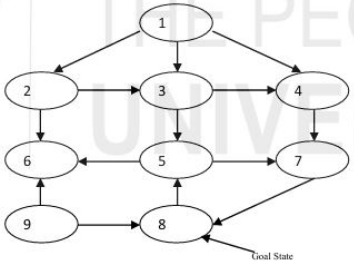
Key Observations

- **Time Complexity**: Most informed search algorithms achieve $O(b^d)$, while uninformed algorithms may reach $O(b^m)$ in worst case.
- **Space Complexity**: Major distinction between algorithms - BFS and A* require exponential space $O(b^d)$, while DFS, IDDFS, IDA*, and RBFS require only linear space $O(bd)$.
- **Completeness**: All algorithms are complete except DFS and Greedy Best-First, which can get stuck in infinite loops or suboptimal paths.
- **Optimality**: Algorithms that consider path costs systematically (BFS with uniform costs, UCS, A* with admissible heuristics) guarantee optimal solutions.

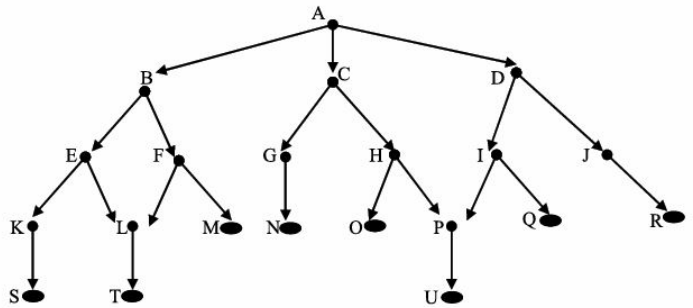
7 Exercises

7.1 Part 1

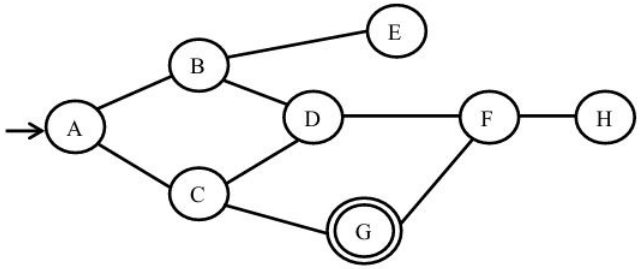
Q.1 Apply BFS and DFS algorithm on the following graph, clearly show the contents of OPEN and CLOSE list.



Q.2 Apply BFS algorithm on the following tree (M is goal node)



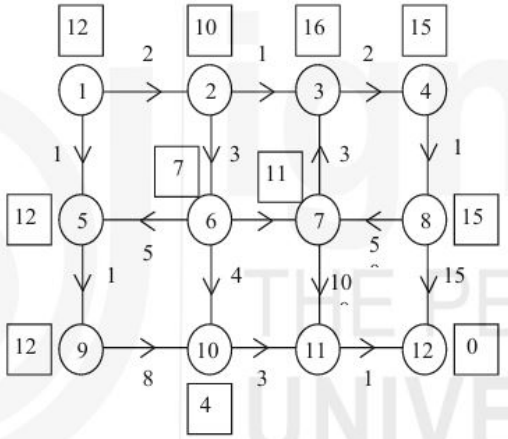
Q.3 Apply BFS algorithm on the following graph.



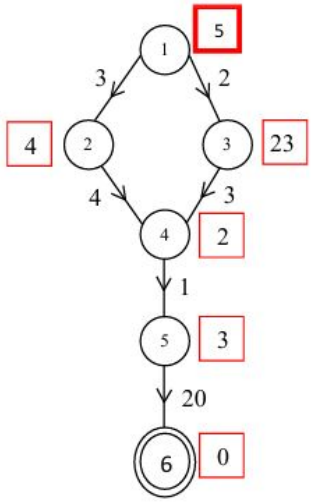
Let A be the state and G be the final or goal state to be searched.

7.2 Part 2

Q.1 Apply A* on the following graph:-

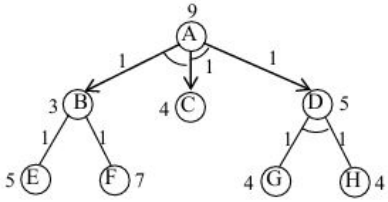


Q.2 Apply A* algorithm on the following graph

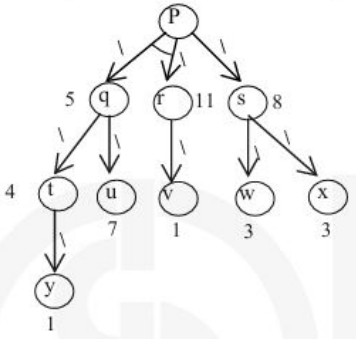


Q.3 Differentiate between the A* and AO* algorithm.

Q.4 Apply AO* algorithm on the following graph. Heuristic value is also given at every node and assume the edge cost value of each node is 1.



Q.5 Apply AO* algorithm on the following graph. Heuristic value is also given at every node and assume the edge cost value of each node is 1.



Q.7 A* algorithm uses f^*=g+h^* to estimate the cost of getting from the initial state to the goal state, where g is a measure of cost getting from initial state to the current node and the function h^* is an estimate of the cost of getting from the current node to the goal state. To find a path involving the fewest number of steps, we should test,

A. g=1

B. g=0

C. h^*=0

D. h^*=1

8 Solutions

8.1 Part 1

Answer 1:

Table 1: Open and closed list for BFS

OPEN LIST	CLOSED LIST
1	1
2,3,4	2
3,4,6	3
4,6,5	4
6,5,7	5
5,7	6
7,	7
8-Goal State	-

Table 2: Open and closed list for DFS

OPEN LIST	CLOSED LIST
1	1
4,3,2	2
4,3,6	6
4,3	3
4,5	5
4,7	7
4,8-Goal State	-

Answer 2

1.	**open = [A]; closed = [ ]**	B is not the goal.
2.	**open = [B,C,D]; closed = [A]**	Put his children onto the queue.
3.	**open = [C,D,E,F}; closed = [B,A]**	Put him in closed. He is done.
4.	**open = [D,E,F,G,H]; closed =[C,B,A]**	Items between red bard are siblings.
5.	**open = [E,F,G,H,I,J]; closed = [D,C,B,A]**	
6.	**open = [F,G,H,I,J,K,L]; closed = [E,D,C,B,A]**	
7.	**open = [G,H,I,J,K,L,M,N]** (as L is already on open); **closed = [G,F,E,D,C,B,A]**	
8.	**open = [H,I,J,K,L,M,N]; closed = [G,F,E,D,C,B,A]**	
9.	and so on until either goal is reached or open is empty.	

Answer 3: Refer BFS section for solution.

8.2 Part 2

Answer 1:

The OPEN and CLOSED list are shown below. Node with their f(n) values are inserted in OPEN list and that node will be expended next whose f(n) value is minimum.

CLOSED

$1^{(12)}$	$2^{(12)}$	$6^{(12)}$	$5^{(13)}$	$10^{(13)}$	$11^{(13)}$	$12^{(13)}$

OPEN

$1^{(12)}$			
$2^{(12)}$	$5^{(13)}$		
$5^{(13)}$	$3^{(14)}$	$6^{(12)}$	
$5^{(13)}$	$3^{(14)}$	$7^{(17)}$	$10^{(13)}$
$3^{(19)}$	$7^{(17)}$	$10^{(13)}$	
$3^{(19)}$	$7^{(17)}$	$10^{(13)}$	$9^{(14)}$
$3^{(19)}$	$7^{(17)}$	$9^{(14)}$	$11^{(13)}$
$3^{(19)}$	$7^{(17)}$	$9^{(14)}$	$12^{(13)}$

Note that only 6 nodes are expended to reach to a goal node. Optimal cost to reach from start state (1) to goal node (12) is **13**.

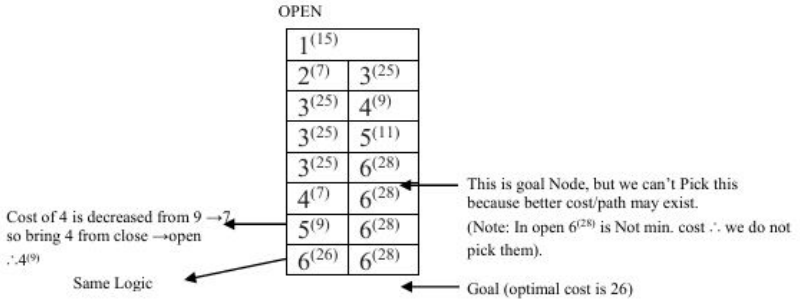
Note: If all the edge cost is positive then **Uniform cost search** (UCS) algorithm is same as Dijkstra's algorithm. Dijkstra algorithm fails if graph is having a negative weight cycle. A* algorithm allows negative weight also. It means A* algorithm work for negative (-ve) edge cost also. If some edge cost is negative, then at any point of successive iteration, we cannot say till that node we have optimum cost (because of the negative cost). So, in this case (-ve edge cost), nodes come back from CLOSED to OPEN.

Answer 2:

The OPEN and CLOSED list are shown below. Node with their f(n) values are inserted in OPEN list and that node will be expended next whose f(n) value is minimum.

CLOSED

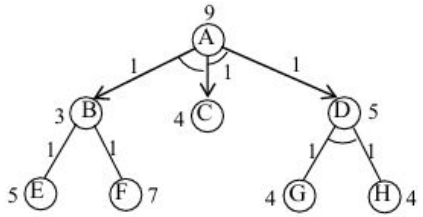
$1^{(15)}$	$2^{(7)}$	$4^{(9)}$	$5^{(11)}$	$3^{(25)}$	$4^{(7)}$	$5^{(9)}$



Optimal cost to reach from start state (1) to goal node (6) is **26**.

Answer 3: An A* algorithm represents an OR graph algorithm that is used to find a single solution (either this or that), but an AO* algorithm represents an AND-OR graph algorithm that is used to find more than one solution by ANDing more than one branch.

Answer 4 :



Path – 1: f(A-B-C) = 1+1+3+4=9

f(B-E) = 1+5=6 f(B-F) = 1+7=8

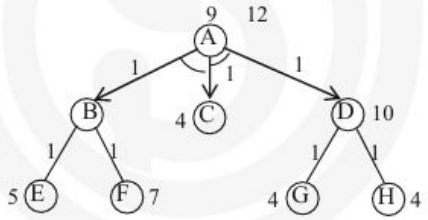
f(A-B-C) = 1+1+6+4=12

Path-2 : f(A-C-D) = 1+1+4+5=11

f(D-G-H) = 1+1+4+4=10

f(A-C-D)= 1+1+4+10=16

AND-OR graph with revised cost is shown in figure.



So, the optimal cost f(A-B-C)= 12

Note that AO* algorithm does not explore all the solution path once it finds a solution.

Answer 5: Similar to Q.4

9 Study Resources

- **GeeksforGeeks – Uninformed Search Algorithms in AI**
  Covers BFS, DFS, Depth-Limited, Iterative Deepening, and Uniform-Cost Search with clear code examples.
  `https://www.geeksforgeeks.org/artificial-intelligence/uniformed-search-algorithms-in-ai/`
- **GeeksforGeeks – Informed Search Algorithms in AI**
  Explains heuristic-based techniques such as A*, Greedy Best-First Search, and others, with implementations.
  `https://www.geeksforgeeks.org/artificial-intelligence/informed-search-algorithms-in-artificial-intelligence/`
- **Medium – AI Search Algorithms: Uninformed vs Informed (with Real-World Examples)**
  Offers intuitive, real-world comparisons of blind versus heuristic search.
  `https://medium.com/ai-simplified-in-plain-english/ai-search-algorithms-uninformed-vs-informed-search-explained-with-real-world-ex`
- **Baeldung – The Informed vs. Uninformed Search Algorithms**
  A well-structured tutorial covering search problem formulation and heuristic strategies.
  `https://www.baeldung.com/cs/informed-vs-uninformed-search`
- **Book – *Artificial Intelligence: A Modern Approach*, 4th ed., by Stuart Russell and Peter Norvig**
  The authoritative and most widely adopted AI textbook—comprehensive coverage of both uninformed and informed search, with rigorous theory, pseudocode, and exercises :contentReference[oaicite:0]index=0.
- **Book – *Artificial Intelligence*, 3rd ed., by Patrick Henry Winston**
  A classic MIT AI textbook with solid treatment of search and reasoning fundamentals, used in MIT's course 6.034 :contentReference[oaicite:1]index=1.
- **YouTube – Gate Smashers "Artificial Intelligence (Complete Playlist)"**
  A comprehensive playlist covering a wide range of AI topics, including search algorithms.
  `https://www.youtube.com/playlist?list=PLxCzCOWd7aiHGhOHV-nwb0HR5US5GFKFI`
- **Textbook – *Search in Artificial Intelligence*, edited by Laveen Kanal and Vipin Kumar**
  A focused collection of state-of-the-art developments in heuristic, AND/OR graph, game tree search, A*, and constraint-satisfaction methods
- **Blog – Data Science Dojo "5 Must-Read Books on AI to Master the Fundamentals"**
  Highlights key AI volumes including "Artificial Intelligence: A Modern Approach" among top picks for fundamental understanding.
- **Textbook – *Introduction to Algorithms* (CLRS), 4th ed., by Cormen, Leiserson, Rivest & Stein**
  Although focused broadly on algorithms, it provides foundational insight into graph search and algorithmic analysis useful for AI search contexts.